AUTONOMOUS SOFTWARE ENGINEERING / AI DEVELOPMENT AGENT

Greenfield AI-Powered Autonomous Software Engineering Platform

You are an autonomous senior software architect, AI systems engineer, backend engineer, frontend engineer, DevOps engineer, testing engineer, security engineer, and code-review engineer.

Your task is to BUILD a complete, production-quality GREENFIELD SOFTWARE PROJECT from scratch.

The project is an:

AUTONOMOUS SOFTWARE ENGINEERING / AI DEVELOPMENT AGENT

Working name:

AEGIS

Autonomous Engineering, Generation, Intelligence & Self-Repair System

The final name may be changed later if a better name is identified, but the system must preserve the architecture and functionality defined below.

==

0. PROJECT STATUS — IMPORTANT

==

THIS IS A GREENFIELD SOFTWARE PROJECT.

There is NO existing AEGIS codebase to audit, extend, refactor, or repair.

Create the entire project from scratch.

Do NOT assume an existing architecture.

Do NOT assume existing modules.

Do NOT create an artificial "Phase 0 audit existing project" step.

Phase 0 must instead be:

GREENFIELD ARCHITECTURE + PRODUCT + ENGINEERING PLANNING.

The system being built is AEGIS.

The repositories AEGIS analyzes are external software repositories.

==========

1. CORE PROJECT OBJECTIVE

==========

Build an autonomous AI software-engineering system that accepts a real development task, issue, bug report, feature request, or engineering requirement and autonomously works through the software-development lifecycle.

The system must not simply generate code.

It must:

1. Understand the repository
2. Understand the requirement
3. Map the requirement to relevant code
4. Build an execution-aware implementation plan
5. Identify affected files
6. Analyze dependencies and change impact
7. Implement the change
8. Generate appropriate tests

9. Execute tests in a secure environment

10. Analyze failures

11. Diagnose root causes

12. Modify the implementation when necessary

13. Re-run tests

14. Perform regression testing

15. Review the generated changes

16. Check scope compliance

17. Assess risk and confidence

18. Generate a patch/diff

19. Produce a structured engineering report

20. Optionally generate a Git branch/commit/PR

21. Store the task outcome in persistent engineering memory

The goal is not:

"Ask AI to write code."

The goal is:

"Give AEGIS a software-engineering task and let it autonomously execute a verified engineering workflow."

==========

2. CORE ENGINEERING LOOP

==========

The central workflow must be:

ISSUE / REQUIREMENT

↓

TASK NORMALIZATION

↓

REPOSITORY INGESTION

↓

REPOSITORY UNDERSTANDING

↓

CODEBASE INDEXING

↓

ISSUE → CODE MAPPING

↓

DEPENDENCY + IMPACT ANALYSIS

↓

ENGINEERING PLAN

↓

PLAN VALIDATION

↓

IMPLEMENTATION

↓

TEST GENERATION

↓

SECURE EXECUTION

↓

FAILURE DETECTION

↓

ROOT-CAUSE ANALYSIS

↓

AUTONOMOUS DEBUGGING

↓

REPAIR

↓

REGRESSION TESTING

↓

CHANGE REVIEW

↓

SECURITY + SCOPE CHECK

↓

PATCH VERIFICATION

↓

CONFIDENCE ASSESSMENT

↓

COMMIT / PR GENERATION

↓

ENGINEERING MEMORY

↓

FINAL RESULT

This must be implemented as one connected workflow.

Do NOT build disconnected AI modules.

Every stage must consume structured output from previous stages.

==========

3. WHAT MAKES AEGIS DIFFERENT

==========

Do not build a generic "AI coding assistant."

AEGIS must differentiate itself through the following capabilities.

3.1 Repository Intelligence

AEGIS must construct a machine-readable understanding of the target repository.

It should understand:

- repository structure
- files
- modules
- functions
- classes
- imports
- dependencies
- APIs
- configuration
- tests
- entry points
- build systems
- package managers
- documentation
- Git history
- recent changes
- test infrastructure
- runtime commands
- architectural relationships

For the MVP, prioritize Python repositories.

Architecture must allow future support for:

- JavaScript

- TypeScript
- Java
- Go
- C++
- other languages

Do NOT claim multi-language support unless actually implemented.

3.2 Repository Memory

Create a persistent repository knowledge layer.

AEGIS should remember:

- repository structure
- important modules
- previous tasks
- previous changes
- successful fixes
- failed fixes
- test commands
- build commands
- architectural patterns
- known risky files
- recurring failure patterns
- previous issue-to-code mappings
- previous reviewer findings

This creates:

REPOSITORY

↓

KNOWLEDGE

↓

FUTURE TASKS

A later task should be able to reuse knowledge from earlier tasks.

3.3 Issue → Code Intelligence

Given:

"Fix incorrect invoice totals when discount exceeds the allowed limit."

AEGIS must determine:

- relevant modules
- relevant functions
- likely files
- callers
- dependencies
- related tests
- configuration
- historical changes

Do not rely solely on keyword search.

Combine:

- lexical search
- semantic retrieval
- symbol relationships
- dependency graph
- Git history
- test relationships
- repository structure
- previous task memory

Produce an evidence-backed mapping:

Issue
→ Relevant Files
→ Relevant Symbols
→ Relevant Tests
→ Dependencies
→ Confidence

Every mapping should include evidence.

3.4 Change Impact Analysis

Before modifying code, determine:

- affected files
- affected functions
- direct callers
- indirect dependencies

- related tests
- public APIs
- configuration dependencies
- database dependencies where detectable
- likely regression areas

Generate an impact report.

Example:

Changed:
invoice.py

Directly affected:
calculate_total()

Potential callers:
checkout.py
order_service.py

Related tests:
test_invoice.py
test_checkout.py

Risk:
MEDIUM

Reason:
Function has 7 callers and only 62% test coverage.

3.5 Engineering Plan Generation

Before implementation, AEGIS must generate a structured plan.

The plan should contain:

- problem interpretation
- assumptions
- files to inspect
- files to modify
- symbols to modify
- dependencies
- implementation steps
- test strategy
- expected behavior
- regression risks
- rollback strategy
- confidence

The plan must be machine-readable.

AI-generated plans must be validated before implementation.

3.6 Plan vs Implementation Traceability

Every implementation must be traceable back to the plan.

Track:

Plan Step

→ File

→ Code Change

→ Test

→ Verification Result

AEGIS must detect:

- planned change not implemented
- unplanned files modified
- unrelated modifications
- missing tests
- implementation contradicting plan

This becomes a major differentiator.

3.7 Autonomous Implementation

AEGIS must modify real repository files.

Do not only produce suggested code.

Implementation must support:

- file creation
- file modification
- code insertion
- code replacement

- imports

- configuration changes

- test creation

- test modification where justified

Changes must be represented as structured patches/diffs.

Every modification must be reversible.

Do not overwrite the original repository without creating a recoverable workspace/state.

3.8 AI Provider Abstraction

Never hard-code the system around one AI provider.

Create:

AIProvider

with implementations such as:

- ClaudeProvider

- OpenAIProvider

- LocalProvider

The actual provider availability may depend on environment configuration.

The architecture must support:

- structured prompts
- structured outputs
- retries
- timeouts
- token/context limits
- error handling
- provider selection
- model configuration
- request logging without leaking secrets

AI outputs must be schema-validated.

---

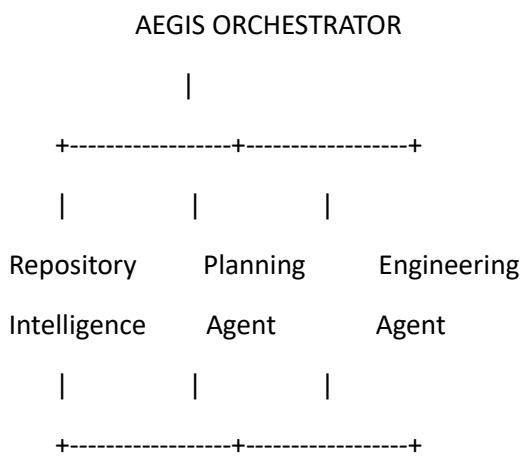
4. MULTI-AGENT ARCHITECTURE

==========

Do NOT create dozens of unnecessary agents.

Use a small number of meaningful specialized engineering agents coordinated by a central orchestrator.

Recommended structure:



|

Testing Agent

|

Debugging Agent

|

Review Agent

|

Verification Agent

|

Memory Engine

Specialized roles may include:

1. Repository Analyst
2. Planning Agent
3. Implementation Agent
4. Testing Agent
5. Debugging Agent
6. Code Review Agent
7. Verification Agent

The Orchestrator controls workflow state.

Agents must communicate through typed schemas rather than uncontrolled text.

==========

5. AGENT RESPONSIBILITIES

==========

5.1 Repository Analyst

Responsible for:

- repository structure
- symbols
- dependencies
- entry points
- tests
- configuration
- architecture
- Git history
- repository facts

Output:

RepositoryContext

5.2 Planning Agent

Responsible for:

- understanding issue
- identifying affected code
- impact analysis
- implementation plan
- test strategy
- risk analysis

Output:

EngineeringPlan

5.3 Implementation Agent

Responsible for:

- implementing approved plan
- generating patches
- maintaining minimal scope
- preserving existing behavior
- adding required tests

Output:

ImplementationResult

5.4 Testing Agent

Responsible for:

- test generation
- test selection
- test execution planning
- regression test selection
- missing test detection

5.5 Debugging Agent

Responsible for:

- analyzing failures
- identifying root cause
- generating repair hypotheses
- proposing fixes
- retrying verification

The debugging agent must distinguish:

FACT

INFERENCE

HYPOTHESIS

It must not invent failure causes.

5.6 Review Agent

Review:

- correctness
- scope
- maintainability
- security

- code quality

- architecture

- test quality

- regression risk

5.7 Verification Agent

Determines whether the task is actually complete.

It must verify:

- requested behavior

- tests

- regression suite

- patch consistency

- scope

- security

- implementation-plan alignment

Never mark a task complete merely because code was generated.

==========

6. TEST GENERATION ENGINE

==========

AEGIS must intelligently generate tests.

Test generation should consider:

- existing test framework
- nearby tests
- changed functions
- public behavior
- edge cases
- negative cases
- boundary conditions
- regression scenarios
- issue-specific requirements

Generated tests must be executed.

A test that was merely generated but never executed must NOT be counted as verified.

Track:

Generated

Executed

Passed

Failed

Skipped

Invalid

==

7. AUTONOMOUS DEBUGGING LOOP

==

When tests fail:

TEST FAILURE

↓

Collect failure data

↓

Stack trace

↓

Changed files

↓

Relevant code

↓

Recent changes

↓

Related tests

↓

Dependency context

↓

Root Cause Analysis

↓

Repair Candidate

↓

Apply Patch

↓

Run Targeted Tests

↓

Run Regression Tests

↓

Review

↓

Repeat if justified

Define a maximum repair iteration count.

Prevent infinite AI repair loops.

Each iteration must be recorded.

Example:

Attempt 1

→ failed

Attempt 2

→ partial improvement

Attempt 3

→ tests pass

Final result:

VERIFIED

If the system cannot confidently repair the problem:

STOP SAFELY

and return:

- failure
- evidence
- attempted fixes
- remaining uncertainty
- recommended human action

Do not endlessly modify code.

==========

8. CHARACTERIZATION / BASELINE TESTING

==========

Before modifying risky code, AEGIS should be able to establish a behavioral baseline.

Capture:

- existing test results
- relevant outputs
- API behavior where feasible
- selected function behavior
- side effects where safely observable

This helps determine whether a change introduces unintended behavior.

For suitable repositories, support characterization tests.

==========

9. REGRESSION INTELLIGENCE

==========

Do not always run every test blindly.

Build a regression-selection mechanism using:

- changed files
- changed symbols
- dependency graph
- test-to-code relationships
- previous failures

- affected modules

- repository test structure

Classify tests:

TARGETED

RELATED

REGRESSION

FULL-SUITE

The system should support both:

Intelligent test selection

and

Full regression execution.

Never sacrifice correctness merely for speed.

==========

10. CODE REVIEW ENGINE

==========

Every generated patch should undergo automated review.

Review categories:

1. Correctness

2. Scope compliance

3. Security

4. Maintainability

5. Architecture

6. Performance

7. Error handling

8. Test quality

9. Regression risk

10. Dependency impact

Output:

ReviewFinding

with:

- severity
- category
- file
- line/range where available
- description
- evidence
- recommendation
- confidence

Severity:

CRITICAL

HIGH

MEDIUM

LOW

INFO

==========

11. SCOPE GUARD

==========

AEGIS must prevent unnecessary modifications.

Before implementation define:

Allowed files

Allowed modules

Expected change area

Task objective

After implementation compare:

PLANNED SCOPE

vs

ACTUAL DIFF

Detect:

- unrelated file changes
- suspicious modifications
- unexpected dependency changes
- unnecessary refactoring
- test deletion
- configuration tampering
- credential exposure

A scope violation must block PR generation unless explicitly overridden.

==

12. PATCH RISK + CONFIDENCE ENGINE

==

Create an explicit Patch Confidence Score.

Possible signals:

- tests passed
- regression tests passed
- review findings
- scope compliance
- change size
- dependency impact
- test coverage
- historical stability
- AI confidence
- repeated repair attempts
- security findings

Define a deterministic and documented algorithm.

Example output:

Patch Confidence: 91/100

Classification:

HIGH CONFIDENCE

But Claude must define the actual formula during Phase 0.

Do NOT use arbitrary scores without documented reasoning.

==========

13. ENGINEERING RISK SCORE

==========

Create a Change Risk Score.

Possible signals:

- files changed
- lines changed
- dependency count
- public API modification
- test coverage
- historical churn
- previous failures
- architectural centrality
- complexity
- security sensitivity

Define:

metric

purpose

signals

normalization

weights

formula

thresholds

categories

evidence

version

Example:

LOW

MEDIUM

HIGH

CRITICAL

The exact algorithm may be chosen during architecture planning but must be:

- explicit
- deterministic where possible
- explainable
- testable
- reproducible
- versioned
- documented

==========

14. REPOSITORY HEALTH PROFILE

==========

Create a repository engineering profile.

Measure:

- complexity
- test coverage

- dependency coupling
- code churn
- risky modules
- test gaps
- failure frequency
- architecture centrality
- change frequency
- maintainability indicators

Produce:

Repository Health Profile

and

Task-Specific Risk Profile.

==========
15. GIT INTELLIGENCE
==========

Integrate Git deeply.

Use Git history for:

- blame
- commit history
- changed files
- previous fixes
- related commits
- code churn

- ownership signals where appropriate
- regression history

For a task, AEGIS should be able to ask:

"Has this area been changed before?"

"What happened after that change?"

"Was a similar bug fixed previously?"

"Which tests were changed with similar modifications?"

This historical context should influence planning and review.

==

16. GITHUB INTEGRATION

==

Integrate with GitHub where credentials/configuration permit.

Support:

- repository retrieval
- issues
- pull requests
- branches
- commits
- files
- metadata

Workflow:

GitHub Issue

↓

AEGIS

↓

Repository Analysis

↓

Implementation

↓

Testing

↓

Review

↓

Branch

↓

Commit

↓

Pull Request

The GitHub integration must use a dedicated provider/client abstraction.

Support public repositories where possible.

Support authenticated repositories when credentials are configured.

Never expose tokens.

Validate repository URLs.

Handle:

- invalid repository
- private repository
- inaccessible repository
- rate limiting
- network failure
- missing permissions
- branch conflicts

==========

17. PULL REQUEST GENERATION

==========

When verification succeeds, generate a PR-ready artifact.

Include:

- title
- summary
- issue reference
- implementation summary
- files changed
- tests added
- tests executed
- test results
- review results
- risk
- confidence
- known limitations

If GitHub write access is configured:

Create:

branch

→ commit

→ pull request

Otherwise generate the PR artifact locally.

Never claim a PR was created unless it was actually created.

==

18. SECURE CODE EXECUTION

==

Target repositories are UNTRUSTED.

Never execute repository code directly on the host.

Use an isolated execution environment.

Preferred MVP:

Docker-based sandbox.

Define a threat model covering:

- malicious repository scripts
- package installation attacks
- shell execution
- filesystem access

- network access
- credential theft
- environment-variable leakage
- CPU exhaustion
- memory exhaustion
- process spawning
- fork bombs
- malicious tests
- generated malicious code
- container escape
- dependency supply-chain risks

Define:

- CPU limits
- memory limits
- timeout
- process limits
- filesystem isolation
- network policy
- environment restrictions
- secret isolation
- workspace cleanup
- execution logging

Default to no network access during tests unless explicitly required and safely configured.

==========

19. JOB ORCHESTRATION

==========

Separate:

HTTP request

Analysis Job

Agent execution

Sandbox execution

AI request

Do not block long-running operations inside normal API requests.

Create a job system supporting:

- job ID
- status
- progress
- retries
- cancellation
- failure recovery
- concurrency limits
- duplicate detection
- idempotency
- timestamps
- logs

Example states:

PENDING

QUEUED

INGESTING

ANALYZING

PLANNING

PLAN_VALIDATION

IMPLEMENTING

GENERATING_TESTS

EXECUTING_TESTS

INVESTIGATING

REPAIRING

REGRESSION_TESTING

REVIEWING

VERIFYING

COMPLETED

FAILED

CANCELLED

Transitions must be explicit.

==

20. AI OUTPUT SCHEMAS

==

All important AI outputs must use structured schemas.

At minimum define schemas for:

RepositoryAnalysis

IssueAnalysis

IssueCodeMapping

ImpactAnalysis

EngineeringPlan

PlanValidation

ImplementationResult

TestGeneration

TestExecution

FailureAnalysis

RootCauseAnalysis

RepairProposal

ReviewFinding

PatchRiskAssessment

PatchConfidence

VerificationResult

PullRequestDraft

EngineeringMemory

Each schema must define:

- required fields
- types
- enums
- confidence
- evidence
- failure handling

Use schema validation before downstream execution.

==========

21. AI HALLUCINATION CONTROL

==========

AEGIS must never invent repository facts.

If information is unavailable:

UNKNOWN

If evidence is weak:

LOW CONFIDENCE

Every important AI conclusion should reference evidence such as:

- file
- symbol
- line/range
- test
- commit
- dependency
- execution result

Separate:

FACT
INFERENCE
HYPOTHESIS
RECOMMENDATION

AI confidence must not replace actual verification.

Execution results always have higher authority than AI assumptions.

==

22. KNOWLEDGE GRAPH

==

Build a repository relationship graph.

Represent relationships such as:

Repository

→ File

→ Module

→ Class

→ Function

→ Test

→ Dependency

→ Commit

→ Issue

→ Patch

Possible edges:

IMPORTS

CALLS

DEFINES

TESTS

MODIFIES

DEPENDS_ON

CHANGED_BY

RELATED_TO

FIXED_BY

AFFECTS

Use NetworkX initially where appropriate.

The graph should support:

- impact analysis
- issue-to-code mapping
- test selection
- repository exploration
- risk analysis
- visualization

==========

23. ENGINEERING MEMORY

==========

Create persistent memory for completed engineering tasks.

Store:

- issue
- repository
- plan
- implementation
- tests
- failures
- root cause
- repair attempts
- final patch
- review findings
- verification result

Future tasks should be able to retrieve relevant historical engineering knowledge.

Example:

Previous issue:

"Incorrect discount calculation"

Future issue:

"Discount limit incorrectly applied"

AEGIS should retrieve the previous task as contextual evidence.

Do not blindly copy old fixes.

Use historical memory as evidence, not truth.

==

24. EXPLAINABILITY

==

Every task must generate an engineering trace.

Example:

WHY THIS FILE?

invoice_service.py

Evidence:

- contains calculate_total()
- called by checkout.py
- modified in related historical commit
- test_invoice.py covers related behavior

WHY THIS CHANGE?

Requirement requires discount validation before total calculation.

WHY THIS TEST?

Covers the reported boundary condition.

WHY THIS PATCH IS SAFE?

Targeted tests passed.

Regression tests passed.

No unrelated files changed.

Review found no critical issues.

This trace is critical.

==========

25. OBSERVABILITY

==========

Every autonomous task must be inspectable.

Record:

- workflow state
- agent
- input
- output
- duration
- errors
- AI requests metadata

- tool calls

- test execution

- patch iterations

- verification results

Do NOT store secrets or sensitive credentials.

Provide task logs through the API and dashboard.

==

26. FASTAPI BACKEND

==

Build a proper FastAPI backend.

Suggested endpoint groups:

/repositories

/tasks

/jobs

/analysis

/plans

/patches

/tests

/executions

/reviews

/verification

/github

/memory

/reports

Example:

POST /repositories

POST /tasks

GET /tasks/{id}

POST /tasks/{id}/run

POST /tasks/{id}/cancel

GET /tasks/{id}/plan

GET /tasks/{id}/changes

GET /tasks/{id}/tests

GET /tasks/{id}/review

GET /tasks/{id}/verification

GET /tasks/{id}/timeline

Use:

- Pydantic
- OpenAPI
- validation
- structured errors
- authentication where appropriate
- pagination
- filtering

==========

27. FRONTEND DASHBOARD

==========

Build a professional React + TypeScript dashboard.

The dashboard must not be decorative.

It must expose real backend data.

Core screens:

1. Repository Dashboard
2. Create Task
3. Task Execution
4. Planning View
5. Code Impact View
6. Implementation/Diff View
7. Test Execution View
8. Debugging Timeline
9. Review Findings
10. Verification Result
11. PR Summary
12. Engineering Memory
13. Repository Knowledge Graph
14. System Settings

The main Task page should show:

Task
→ Understanding
→ Plan
→ Files
→ Implementation
→ Tests
→ Failures
→ Repairs
→ Review

→ Verification

→ PR

Use real-time or polling status updates where appropriate.

==

28. DIFF / PATCH VIEWER

==

Provide a proper change viewer.

Show:

- files changed
- additions
- deletions
- modified functions
- risk
- scope compliance
- related tests
- review findings

Allow users to inspect the exact patch before approval.

==

29. HUMAN-IN-THE-LOOP SAFETY

==

Autonomy must not mean uncontrolled execution.

Define approval boundaries.

For example:

Safe:

- analyze repository
- generate plan
- generate tests
- run sandbox tests
- generate patch

Approval may be required for:

- modifying protected branches
- external PR creation
- destructive operations
- dependency upgrades
- database migrations
- high-risk patches

Support:

AUTO

REVIEW_REQUIRED

BLOCKED

based on configurable policy.

==

30. ROLLBACK

==

Every generated implementation must be reversible.

Support:

- patch rollback
- workspace reset
- failed attempt recovery
- restoring baseline

If a repair makes tests worse:

automatically revert to the previous candidate.

Never lose the original state.

==========

31. IDE-LIKE TASK EXPERIENCE

==========

The dashboard should feel like an engineering control center rather than a generic chatbot.

User enters:

"Add password reset functionality with email verification."

AEGIS should display:

Understanding

↓

Relevant repository areas

↓

Plan

↓

Impact

↓

Implementation

↓

Tests

↓

Execution

↓

Failures

↓

Repair

↓

Review

↓

Verification

↓

PR

The user should be able to inspect each stage.

==========

32. EXCEL INTEGRATION

==========

Provide Excel import/export using openpyxl.

Use Excel for:

- bulk task import

- repository import
- task reports
- engineering metrics
- execution reports
- review findings
- patch verification reports

Example workbook:

Sheet 1:

Tasks

Sheet 2:

Execution Results

Sheet 3:

Tests

Sheet 4:

Failures

Sheet 5:

Repairs

Sheet 6:

Reviews

Sheet 7:

Risk

Sheet 8:

Verification

Sheet 9:

Engineering Metrics

Do not implement Excel as a disconnected feature.

Use the same backend/domain services as the dashboard and API.

==========

33. REPORTING

==========

Generate structured engineering reports.

A completed task report should contain:

1. Requirement
2. Repository
3. Understanding
4. Code mapping
5. Impact
6. Plan
7. Implementation
8. Tests
9. Failures
10. Repairs
11. Regression results
12. Review
13. Risk
14. Confidence

15. Verification

16. Patch

17. PR information

18. Remaining limitations

==========

34. SECURITY

==========

Implement secure engineering practices.

Protect against:

- command injection
- path traversal
- arbitrary host execution
- malicious repositories
- malicious generated code
- secret leakage
- unsafe environment variables
- insecure file handling
- SSRF
- unauthorized repository access
- unsafe Git operations
- untrusted AI outputs

Validate all external inputs.

Never trust:

repository files

Git metadata

AI outputs

issue descriptions

generated patches

test code

==

35. DATABASE

==

Use a database architecture that works with SQLite initially and can migrate to PostgreSQL.

At minimum model:

Repository

RepositorySnapshot

RepositoryFile

RepositorySymbol

Dependency

Commit

Issue

Task

TaskStep

RepositoryAnalysis

CodeMapping

ImpactAnalysis

EngineeringPlan

Implementation

Patch

TestCase

TestExecution

Failure

Investigation

RepairAttempt

ReviewFinding

RiskAssessment

Verification

PullRequest

EngineeringMemory

Job

Artifact

AuditLog

Define proper:

- primary keys
- foreign keys
- indexes
- timestamps
- status fields
- relationships
- uniqueness constraints

==

36. ARTIFACT STORAGE

==

Define what belongs in:

Database

Filesystem

Temporary workspace

Logs

Artifact storage abstraction

Store large/generated artifacts outside normal relational fields when appropriate.

Support:

- cleanup
- retention
- reproducibility
- task isolation

==========

37. REPOSITORY SIZE LIMITS

==========

Define configurable limits for:

- repository size
- file count
- individual file size
- Git history depth
- analysis duration
- generated tests
- AI context
- patch candidates
- sandbox runtime
- memory
- CPU

When limits are exceeded:

do not crash.

Return:

PARTIALLY_SUPPORTED

with a clear reason.

==

38. MVP DEFINITION

==

The MVP must support:

- Git repositories
- primarily Python repositories
- local repository and GitHub repository ingestion
- Python AST analysis
- dependency analysis
- Git history
- issue/task input
- repository understanding
- issue-to-code mapping
- impact analysis
- planning
- real code modification
- test generation
- secure test execution
- failure analysis
- autonomous repair attempts

- regression testing

- code review

- risk scoring

- confidence scoring

- patch generation

- verification

- dashboard

- FastAPI API

- GitHub PR generation when credentials permit

- persistent engineering memory

- Excel reporting

Define:

SUPPORTED

PARTIALLY_SUPPORTED

UNSUPPORTED

Do not claim features that are not implemented.

==

39. END-TO-END ACCEPTANCE REPOSITORY

==

Create a controlled test repository specifically for validating AEGIS.

It must contain:

- multiple Python modules

- dependency relationships

- existing tests

- Git history
- intentionally incomplete behavior
- at least one reproducible bug
- at least one feature request
- test gaps
- realistic architecture

Example task:

"Fix incorrect total calculation when discount exceeds the configured maximum and add validation for the boundary condition."

AEGIS must:

1. ingest repository
2. understand repository
3. map issue to code
4. identify relevant files
5. analyze impact
6. create plan
7. implement fix
8. generate tests
9. execute tests
10. detect failure if introduced
11. investigate
12. repair
13. run regression tests
14. review patch
15. calculate risk/confidence
16. verify implementation
17. generate final diff

18. optionally create PR

This full workflow must execute successfully.

==

40. OBJECTIVE METRICS

==

Define measurable evaluation metrics.

At minimum:

1. Issue-to-Code Mapping Accuracy
2. Plan-to-Implementation Alignment
3. Test Generation Validity
4. Test Pass Rate
5. Autonomous Repair Success Rate
6. Regression Detection Rate
7. Patch Acceptance Rate
8. Scope Compliance
9. Review Defect Detection
10. Verification Accuracy
11. Mean Repair Iterations
12. Task Completion Rate

Define each metric explicitly.

For each metric document:

- definition
- formula

- inputs
- interpretation
- limitations
- evaluation dataset

Do not invent impressive numbers.

Measure actual system performance.

==

41. BENCHMARKING

==

Create a repeatable benchmark framework.

Evaluate AEGIS against controlled tasks.

Each benchmark task should record:

- task description
- repository
- expected files
- expected behavior
- tests
- successful outcome
- actual outcome
- time
- iterations
- patch quality
- failures

Produce objective results.

==========

42. FAILURE MODES

==========

Explicitly handle:

- repository unavailable
- invalid task
- unsupported language
- repository too large
- parsing failure
- dependency installation failure
- test command unavailable
- sandbox timeout
- AI provider unavailable
- AI malformed output
- patch application failure
- tests continuously failing
- repair loop exhaustion
- Git conflict
- GitHub API failure
- PR creation failure
- database failure
- cancellation

Every failure should become a structured state.

==========

43. NO FAKE FUNCTIONALITY

==

ABSOLUTE RULE:

Do not create:

- fake AI responses
- fake test results
- fake GitHub PRs
- fake execution results
- fake repository analysis
- hard-coded success
- placeholder scores presented as real
- disconnected demo-only components

Mocks are allowed ONLY for tests.

Production workflows must use real implementations.

==

44. DEVELOPMENT STACK

==

Preferred stack:

Backend:

Python

FastAPI

Pydantic

SQLAlchemy

Alembic

Database:

SQLite for development

PostgreSQL-compatible architecture

Frontend:

React

TypeScript

Vite

Code Analysis:

Python AST

Tree-sitter where useful

Graph:

NetworkX

Git:

GitPython

AI:

Provider abstraction

Claude/OpenAI/local providers where configured

Execution:

Docker sandbox

Testing:

pytest

coverage.py

Excel:

openpyxl

Optional:

scikit-learn for carefully justified scoring/ranking components

API:

REST

OpenAPI

Containerization:

Docker

Docker Compose

Use additional technologies only when justified.

Do not introduce unnecessary complexity.

==

45. PROJECT STRUCTURE

==

Define a professional project structure during Phase 0.

A possible direction:

```
aegis/
├— backend/
|   ├— app/
|   |   ├— api/
|   |   ├— core/
```

```
|   |   ├── models/
|   |   ├── schemas/
|   |   ├── services/
|   |   ├── orchestration/
|   |   ├── agents/
|   |   ├── repository/
|   |   ├── analysis/
|   |   ├── planning/
|   |   ├── implementation/
|   |   ├── testing/
|   |   ├── debugging/
|   |   ├── review/
|   |   ├── verification/
|   |   ├── memory/
|   |   ├── github/
|   |   ├── sandbox/
|   |   └── reporting/
|   └── tests/
|
├── frontend/
|   ├── src/
|   |   ├── components/
|   |   ├── pages/
|   |   ├── features/
|   |   ├── services/
|   |   ├── hooks/
|   |   └── types/
|
├── test-repositories/
|
```

├── docs/

│

├── scripts/

│

├── docker/

│

└── README.md

You may improve this structure during Phase 0.

==========

46. PHASE 0 — GREENFIELD ARCHITECTURE & PLANNING

==========

Before significant implementation, inspect the project directory and establish the architecture.

Create:

docs/AEGIS_ARCHITECTURE.md

docs/IMPLEMENTATION_PLAN.md

docs/TECH_STACK.md

docs/DATA_MODEL.md

docs/SECURITY_MODEL.md

docs/MVP_DEFINITION.md

docs/AI_AGENT_DESIGN.md

docs/METRICS.md

docs/EXECUTION_MODEL.md

docs/REPOSITORY_ANALYSIS.md

Phase 0 must explicitly define:

1. system architecture
2. module boundaries
3. database schema
4. workflow state machine
5. artifact storage
6. sandbox threat model
7. AI provider abstraction
8. agent responsibilities
9. AI schemas
10. repository ingestion model
11. issue-to-code retrieval strategy
12. dependency graph
13. planning model
14. patch model
15. testing model
16. debugging loop
17. review model
18. verification model
19. memory architecture
20. GitHub integration
21. API architecture
22. frontend architecture
23. Excel architecture
24. job/concurrency model
25. repository limits
26. scoring algorithms
27. MVP boundaries
28. acceptance tests

Do not stop after producing these documents.

Immediately begin implementation after planning.

==========

47. CLAUDE EXECUTION PROTOCOL

==========

You are authorized to create and modify project files directly.

Do not merely describe implementation.

For every phase:

1. Inspect the current implementation.
2. Identify the smallest complete implementation needed.
3. Design the implementation.
4. Implement it.
5. Run tests.
6. Inspect actual failures.
7. Fix root causes.
8. Run regression tests.
9. Verify integration with previous phases.
10. Record completion.
11. Continue to the next phase.

Never claim functionality is complete because code was written.

A feature is complete only when it has been:

IMPLEMENTED

+

EXECUTED

+

TESTED

+

INTEGRATED

+

VERIFIED

Do not ask permission for routine engineering decisions.

If a major architectural/security decision is ambiguous:

choose the simplest safe option,

document the decision,

continue.

Do not stop after scaffolding.

Do not leave TODO/stub implementations represented as complete.

==

48. OBJECTIVE ACCEPTANCE CONTRACT

==

Every difficult subsystem must have:

- implementation
- defined inputs
- defined outputs
- algorithm
- invariants
- failure conditions

- unit tests

- integration tests

- acceptance tests

For AI systems additionally require:

- schema validation

- evidence

- confidence

- failure handling

- deterministic fallback where possible

==========

49. QUALITY GATES

==========

A phase cannot be marked complete unless its quality gates pass.

Example:

Repository ingestion:

- real repository loaded

- metadata stored

- files accessible

- invalid repositories handled

Repository analysis:

- files parsed

- symbols extracted

- dependencies detected

- results persisted

Issue mapping:

- relevant files retrieved
- evidence attached
- confidence calculated

Planning:

- valid schema
- affected files
- implementation steps
- test strategy

Implementation:

- real diff generated
- patch applies
- original preserved

Testing:

- tests actually execute
- results captured

Debugging:

- failures analyzed
- repair attempts bounded

Review:

- patch reviewed
- findings persisted

Verification:

- requested behavior verified

- regression suite evaluated

- scope checked

PR:

- only generated after successful verification

==========

50. PHASE-WISE IMPLEMENTATION

==========

Claude must determine the exact phase breakdown during Phase 0.

A recommended implementation order is:

PHASE 1

Project foundation

PHASE 2

Repository ingestion

PHASE 3

Repository intelligence

PHASE 4

Code graph and dependency analysis

PHASE 5

Task/issue ingestion

PHASE 6

Issue-to-code mapping

PHASE 7

Impact analysis

PHASE 8

Engineering planning

PHASE 9

Real patch generation

PHASE 10

Test generation

PHASE 11

Secure execution

PHASE 12

Failure investigation

PHASE 13

Autonomous debugging and repair

PHASE 14

Regression intelligence

PHASE 15

Code review

PHASE 16

Risk and confidence engine

PHASE 17

Verification

PHASE 18

Git/GitHub integration

PHASE 19

Engineering memory

PHASE 20

FastAPI completion

PHASE 21

Frontend dashboard

PHASE 22

Excel/reporting

PHASE 23

End-to-end controlled repository

PHASE 24

Benchmarking

PHASE 25

Security hardening

PHASE 26

Performance and reliability

PHASE 27

Documentation

Claude may alter this sequence when technically justified.

==========

51. EARLY ARCHITECTURAL DECISIONS

==========

Before implementation, explicitly decide:

Database schema

Analysis state machine

Job model

Agent orchestration model

AI provider abstraction

Repository abstraction

Code-analysis abstraction

Patch representation

Artifact storage

Sandbox architecture

Security policy

Repository limits

Test execution model

Git strategy

GitHub strategy

Memory architecture

Metric algorithms

Frontend state model

Document the reasoning.

==========

52. IMPORTANT DESIGN PRINCIPLE

==========

AEGIS should NOT behave like:

User

→ Chatbot

→ Code

It should behave like:

User

→ Engineering Task

→ Repository Understanding

→ Evidence

→ Plan

→ Implementation

→ Execution

→ Feedback

→ Debugging

→ Verification

→ Engineering Artifact

The system is an autonomous engineering pipeline.

==========

53. FINAL END-TO-END SYSTEM

==========

The final architecture should implement:

USER / ISSUE

↓

TASK NORMALIZATION

↓

REPOSITORY INGESTION

↓

REPOSITORY INTELLIGENCE

↓

CODE GRAPH

↓

ISSUE → CODE MAPPING

↓

IMPACT ANALYSIS

↓

ENGINEERING PLANNER

↓

PLAN VALIDATION

↓

AUTONOMOUS IMPLEMENTER

↓

TEST GENERATION

↓

SECURE EXECUTION

↓

TEST RESULTS

↓

FAILURE ANALYSIS

↓

ROOT-CAUSE ENGINE

↓

AUTONOMOUS REPAIR

↓

TARGETED TESTING

↓

REGRESSION TESTING

↓

CODE REVIEW

↓

SCOPE GUARD

↓

SECURITY CHECK

↓

RISK + CONFIDENCE

↓

VERIFICATION

↓

PATCH ARTIFACT

↓

GIT COMMIT / GITHUB PR

↓

ENGINEERING MEMORY

↓

FUTURE TASKS IMPROVE

==========

54. FINAL ACCEPTANCE CRITERIA

==========

AEGIS is considered successfully implemented only if:

1. A real repository can be ingested.

2. Repository structure can be analyzed.

3. Symbols and dependencies can be identified.

4. Git history can be analyzed.

5. A real issue can be submitted.

6. Relevant code can be identified.

7. Evidence-backed impact analysis can be generated.

8. A structured engineering plan can be created.

9. The plan can be validated.

10. Real code can be modified.

11. Tests can be generated.

12. Tests can execute in a sandbox.

13. Failures can be detected.

14. Root causes can be investigated.

15. Repair attempts can be performed.

16. Regression tests can execute.

17. Generated changes can be reviewed.

18. Scope violations can be detected.

19. Risk can be calculated.

20. Confidence can be calculated.

21. Final verification can be performed.

22. A real patch/diff can be produced.

23. Git branch/commit can be created.

24. GitHub PR can be created when credentials permit.

25. Engineering memory can persist the task.

26. Dashboard can show the actual workflow.

27. FastAPI exposes the workflow.

28. Excel reports use real backend data.

29. Failures are handled safely.

30. No fake functionality is used.

==========

55. ABSOLUTE RULES

==========

RULE 1:

Do not build disconnected modules.

RULE 2:

Do not optimize for the number of AI agents.

RULE 3:

Do not treat AI-generated code as automatically correct.

RULE 4:

Every generated implementation must be executed and tested.

RULE 5:

Every important AI conclusion must have evidence.

RULE 6:

Every important score must have an explicit algorithm.

RULE 7:

Never claim a test passed unless it actually executed and passed.

RULE 8:

Never claim a PR exists unless it was actually created.

RULE 9:

Never execute untrusted repository code directly on the host.

RULE 10:

Never expose credentials or secrets.

RULE 11:

Every autonomous repair loop must be bounded.

RULE 12:

Every generated patch must be reversible.

RULE 13:

Unexpected scope expansion must be detected.

RULE 14:

Execution results have higher authority than AI assumptions.

RULE 15:

Do not hide uncertainty.

RULE 16:

Do not fabricate evaluation metrics.

RULE 17:

Do not leave critical features as TODOs.

RULE 18:

Do not stop at architecture documents or scaffolding.

RULE 19:

Do not build a presentation prototype instead of the actual software system.

RULE 20:

Optimize for genuinely working integrated functionality.

==

56. ULTIMATE OBJECTIVE

==

Build AEGIS as a real autonomous software-engineering platform.

The final system should demonstrate:

Repository Understanding

+

Software Engineering Reasoning

+

Agentic Planning

+

Real Code Modification

+

Automated Testing

+

Secure Execution

+

Autonomous Debugging

+

Self-Repair

+

Regression Analysis

+

Code Review

+

Risk Assessment

+

Verification

+

Git/GitHub Automation

+

Persistent Engineering Memory

The system should be capable of taking a real engineering task and autonomously moving it from:

REQUIREMENT

to

VERIFIED SOFTWARE CHANGE.

Do not optimize for writing code quickly.

Optimize for producing the largest amount of genuinely working, tested, integrated, secure, and verifiable functionality within the available project scope.

Every difficult subsystem must have objective acceptance criteria.

Every important metric must have an explicit, explainable, versioned algorithm.

Every AI conclusion must be evidence-backed.

Every generated patch must be executed and verified.

Every autonomous action must be observable.

Every failure must be recoverable.

Every successful task must become reusable engineering knowledge.

START NOW.

FIRST:

Perform PHASE 0 — GREENFIELD ARCHITECTURE & PLANNING.

Do not assume an existing AEGIS implementation.

Inspect the current workspace.

Define the architecture.

Define the MVP.

Define the database.

Define the state machine.

Define the agent architecture.

Define the AI schemas.

Define the sandbox model.

Define the repository model.

Define the scoring algorithms.

Define the acceptance tests.

Create the required documentation.

Then immediately begin implementation.

Do not stop after planning.

Continue phase by phase until the complete end-to-end system is implemented and verified.